

Scenario 7 — Underwriter Manual Override, Bypassing the Oracle Entirely

Full mechanics: see Step-3 for claim submission. This scenario replaces Step-4 with a different function entirely — `ClaimRegistry` has two independent paths to change a claim's status, and this is the one none of the other scenarios exercise.

Values used

	Value
Policyholder	Alice, 0x1111...aaaa
templateId	3 (Premium)
Claim amount	221 ether
Path taken	<code>ClaimRegistry.setClaimStatus</code> directly, called by the Underwriter Safe — not <code>OracleAdapter.submitVerification</code>
Decision	Rejected, by human judgment, not the mock oracle logic

Walkthrough (deltas)

- **Steps 1-3:** identical to Scenario 3 — claim 1, status Submitted.
- **Step 4 is skipped entirely.** No call to the backend's `trigger-verification` endpoint, no call to `oracle-service`, no EIP-712 signature is ever produced for this claim.
- Instead, the Underwriter Safe (0x7777...aaaa) directly calls:

```
ClaimRegistry.setClaimStatus(uint256 claimId, ClaimStatus status)
```

- **Input:** `claimId = 1`, `status = Rejected` (enum value 4)
- **Caller check:** `onlyRole(UNDERWRITER_ROLE)` — same role that calls `payoutClaim` in the normal flow, but this is a *different* function, with no oracle involvement anywhere in its logic.
- **Traversal:** checks `claims[1]` exists (`claimant != address(0)`); sets `claims[1].status = Rejected`; since `Rejected` is a terminal status, sets `claims[1].processedAt = block.timestamp`; emits `ClaimStatusChanged(1, Rejected)`.
- **Output:** none (void).

Why this path exists at all

Not every claim is suited to automated, oracle-driven verification — the mock oracle's placeholder logic (a flat dollar threshold) stands in for what would, in a real system, be actual flight-status data; but plenty of real insurance claims need human judgment regardless of how sophisticated the automated check is. `setClaimStatus` is that escape hatch: any claim a human underwriter reviews can be approved or rejected directly, with `recordOracleVerification` (Step 4's function) simply being the *other*, automated way to reach the same two possible end states (`Approved` / `Rejected`). Both paths write to the exact same `claims[claimId].status` field — from `ClaimRegistry`'s own state's perspective, there's no difference between "the oracle approved this" and "an underwriter approved this by hand."

End state

```
claims[1].status: Rejected (reached via setClaimStatus, not recordOracleVerification)
usedOracleRequestIds: untouched - no requestId was ever generated or consumed for this
claim
```

If the underwriter had instead called `setClaimStatus(1, Approved)` (enum value 2), `payoutClaim` would then work exactly as in Step-5 — `payoutClaim` only checks `status == Approved`, and doesn't care or know whether that status was reached via the oracle or via this direct path.